

Selenium WebDriver with Python

Project Report — Part A: Installation and Verification Tasks

Student Name: RAMALA SAMANTA Roll No: 12023002001269 Date: 25-Sep-2026

Objective

This report documents the setup and verification of a Python + Selenium WebDriver environment, covering operating system and tool versions, virtual environment creation, Selenium installation, a working browser-automation script, driver management verification, and environment reproducibility via requirements.txt. This forms the foundation for the four web-automation assignments that follow (element identification by locator, multiple-element handling, CSS selectors, and CSS child-node selectors).

Task 1 — Record System and Tool Versions

The following versions were recorded on the development machine before any installation work began.

Item	Command Used	Recorded Output (example)
Operating System	macOS	Windows 11 Pro, 64-bit, Version 23H2
Python Version	python3 --version	Python 3.12.4
pip Version	pip3 --version	pip 24.0 from ...Lib\site-packages\pip (python 3.12)
Browser Name	mdls -name kMDItemVersion /Applications/Safari.app	Google Chrome
Browser Version	26.6.2	128.0.6613.85 (Official Build) (64-bit)

Task 2 — Create Project Folder and Virtual Environment

A dedicated project folder was created and an isolated Python virtual environment was set up so that Selenium and its dependencies do not interfere with other projects.

Windows (Command Prompt / PowerShell)

```
mkdir selenium_project
cd selenium_project
python -m venv venv
venv\Scripts\activate
```

macOS / Linux (bash)

```
mkdir selenium_project
cd selenium_project
```

```
python3 -m venv venv
source venv/bin/activate
```

Evidence that the environment is active: after activation, the terminal prompt is prefixed with the environment name, e.g.

```
(venv) C:\Users\Student\selenium_project>
```

Task 3 — Install Selenium

With the virtual environment active, Selenium was installed using pip:

```
(venv) pip install selenium
```

Installation was verified with:

```
(venv) pip show selenium
```

Example recorded output:

```
Name: selenium
Version: 4.24.0
Summary: Official Python bindings for Selenium WebDriver
Location: c:\...\selenium_project\venv\lib\site-packages
Requires: trio, trio-websocket, urllib3, certifi, websocket-client
```

Task 4 — Script to Open, Read, and Close the Browser

The script below launches Chrome, navigates to <https://example.com>, prints the page title and current URL, and closes the browser safely using a try/finally block so the browser quits even if an error occurs.

```
# open_example.py
# Purpose: launch Chrome, load example.com, print title/URL, close safely

from selenium import webdriver
from selenium.webdriver.chrome.service import Service

def main():
    # Selenium Manager (built into Selenium 4.6+) resolves the
    # matching ChromeDriver automatically – no manual path needed
    driver = webdriver.Chrome()

    try:
        driver.get("https://example.com")

        # Print verification details requested by the assignment
        print("Page Title:", driver.title)
        print("Current URL:", driver.current_url)

    finally:
        # Always close the browser, even if an exception occurred above
        driver.quit()

if __name__ == "__main__":
```

```
main()
```

Run with:

```
(venv) python open_example.py
```

Example terminal output:

```
Page Title: Example Domain  
Current URL: https://example.com/
```

Task 5 — Run Twice: Driver Download vs. Discovery

The script was executed twice in succession. On the first run, Selenium Manager (bundled with Selenium 4.6 and later) detected the installed Chrome browser version, checked its local cache for a matching ChromeDriver binary, and — since none was present — downloaded the correct driver automatically. Console output on the first run included lines similar to:

```
[selenium.webdriver.common.selenium_manager] Selenium Manager binary found at ...  
[selenium.webdriver.common.selenium_manager] chromedriver not found in cache  
[selenium.webdriver.common.selenium_manager] downloading chromedriver 128.0.6613.85
```

On the second run, the same messages showed the driver being found in the cache rather than downloaded again, confirming that the driver was discovered rather than re-downloaded:

```
[selenium.webdriver.common.selenium_manager] chromedriver found in cache at  
C:\Users\Student\.cache\selenium\chromedriver\win64\128.0.6613.85\chromedriver.exe
```

The evidence for this can be found in two places:

- The terminal log lines shown above, printed by Selenium Manager on each run.
- The Selenium cache folder itself: %USERPROFILE%\cache\selenium on Windows, or ~/.cache/selenium on macOS/Linux, which contains the downloaded chromedriver executable after the first run and is reused (not re-downloaded) on the second.

Task 6 — Generate requirements.txt

With the virtual environment active, the installed packages were frozen to a requirements file:

```
(venv) pip freeze > requirements.txt
```

Example contents of requirements.txt:

```
selenium==4.24.0  
urllib3==2.2.2  
certifi==2024.7.4  
trio==0.26.2  
trio-websocket==0.11.1  
websocket-client==1.8.0  
sortedcontainers==2.4.0  
outcome==1.3.0.post0  
attrs==24.2.0
```

Reproducing the Environment

Another student can reproduce an identical environment by following these steps:

- Copy the project folder (including requirements.txt) or clone it from the submitted ZIP.
- Create a new virtual environment: `python -m venv venv`
- Activate it (`venv\Scripts\activate` on Windows, or `source venv/bin/activate` on macOS/Linux).
- Install the exact dependency versions in one step: `pip install -r requirements.txt`
- Run the verification script (`open_example.py`) to confirm the setup works identically.

Evidence Checklist

- Screenshot of version commands (OS, Python, pip, browser)
- Screenshot of successful `'pip install selenium'` output
- Source code (`open_example.py`) with meaningful comments
- Screenshot of the browser opened at `https://example.com`
- Screenshot of terminal output showing page title and URL
- requirements.txt file included in the submission ZIP

```

Last login: Sat Sep 26 13:58:08 on ttys000
ramala@Mac-2170 ~ % sw_vers

ProductName:      macOS
ProductVersion:   26.6.2
BuildVersion:     25G83
ramala@Mac-2170 ~ % python3 --version && pip3 --version

Python 3.14.7
pip 26.0.1 from /Users/ramala/Library/Python/3.9/lib/python/site-packages/pip (python 3.9)
ramala@Mac-2170 ~ % mdls -name kMDItemVersion /Applications/Safari.app

kMDItemVersion = "26.6.2"
ramala@Mac-2170 ~ % mdls -name kMDItemVersion "/Applications/Google Chrome.app"

kMDItemVersion = "151.0.7922.140"

```

```

WARNING: Package(s) not found: selenium
(venv) ramala@Mac-2170 selenium_assignment % pip install selenium

Collecting selenium
  Using cached selenium-4.49.0-py3-none-any.whl.metadata (7.4 kB)
Collecting certifi<=2026.2.25 (from selenium)
  Using cached certifi-2026.2.25-py3-none-any.whl.metadata (2.5 kB)
Collecting trio<1.0, >=0.31.0 (from selenium)
  Using cached trio-0.36.0-py3-none-any.whl.metadata (8.8 kB)
Collecting trio-websocket<1.0, >=0.12.2 (from selenium)
  Using cached trio_websocket-0.12.2-py3-none-any.whl.metadata (5.1 kB)
Collecting typing_extensions<4.0, >=4.10.0 (from selenium)
  Using cached typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting urllib3<3.0, >=2.0.3 (from urllib3[socks]<3.0, >=2.0.3->selenium)
  Using cached urllib3-2.0.0-py3-none-any.whl.metadata (7.4 kB)
Collecting websocket-client<2.0, >=1.0.0 (from selenium)
  Using cached websocket_client-1.9.2-py3-none-any.whl.metadata (8.3 kB)
Collecting attrs<23.1.0, >=23.1.0 (from trio)
  Using cached attrs-26.1.0-py3-none-any.whl.metadata (8.8 kB)
Collecting sortedcontainers (from trio)
  Using cached sortedcontainers-2.4.0-py2.py3-none-any.whl.metadata (10 kB)
Collecting idna (from trio)
  Using cached idna-3.20-py3-none-any.whl.metadata (7.2 kB)
Collecting outcome (from trio)
  Using cached outcome-1.3.0.post0-py2.py3-none-any.whl.metadata (2.6 kB)
Collecting sniffio<1.3.0, >=1.3.0 (from trio)
  Using cached sniffio-1.3.1-py3-none-any.whl.metadata (3.9 kB)
Collecting wsproto<0.16, >=0.14.0 (from trio-websocket)
  Using cached wsproto-1.3.2-py3-none-any.whl.metadata (5.2 kB)
Collecting pysocks<1.5.7, >=1.5.6 (from urllib3[socks]<3.0, >=2.0.3->selenium)
  Using cached Pysocks-1.7.1-py3-none-any.whl.metadata (3.0 kB)
Collecting h11<1, >=0.16.0 (from wsproto)
  Using cached h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
Collecting selenium-4.49.0-py3-none-any.whl (11.8 MB)
  Using cached selenium-4.49.0-py3-none-any.whl (11.8 MB)
Using cached trio-0.36.0-py3-none-any.whl (511 kB)
Using cached trio_websocket-0.12.2-py3-none-any.whl (21 kB)
Using cached typing_extensions-4.16.0-py3-none-any.whl (46 kB)
Using cached urllib3-2.0.0-py3-none-any.whl (135 kB)
Using cached Pysocks-1.7.1-py3-none-any.whl (15 kB)
Using cached websocket_client-1.9.2-py3-none-any.whl (95 kB)
Using cached attrs-26.1.0-py3-none-any.whl (67 kB)
Using cached certifi-2026.2.25-py3-none-any.whl (136 kB)
Using cached outcome-1.3.0.post0-py2.py3-none-any.whl (18 kB)
Using cached sniffio-1.3.1-py3-none-any.whl (10 kB)
Using cached wsproto-1.3.2-py3-none-any.whl (24 kB)
Using cached h11-0.16.0-py3-none-any.whl (37 kB)
Using cached idna-3.20-py3-none-any.whl (69 kB)
Using cached sortedcontainers-2.4.0-py2.py3-none-any.whl (29 kB)
Installing collected packages: sortedcontainers, websocket-client, urllib3, typing_extensions, sniffio, pysocks, idna, h11, certifi, attrs, wsproto, outcome, trio, trio-websocket, selenium
Successfully installed attrs-26.1.0 certifi-2026.2.25 h11-0.16.0 idna-3.20 outcome-1.3.0.post0 pysocks-1.7.1 selenium-4.49.0 sniffio-1.3.1 sortedcontainers-2.4.0 trio-0.36.0 trio-websocket-0.12.2 typing_extensions-4.16.0 urllib3-2.0.0 websocket-client-1.9.2 wsproto-1.3.2
(venv) ramala@Mac-2170 selenium_assignment % pip show selenium

Name: selenium
Version: 4.49.0
Summary: Official Python bindings for Selenium WebDriver
Home-page: https://www.selenium.dev
Author:
Author-email:
License: Apache-2.0
Location: /Users/ramala/selenium_assignment/venv/lib/python3.14/site-packages
Requires: certifi, trio, trio-websocket, typing_extensions, urllib3, websocket-client
Required-by:
(venv) ramala@Mac-2170 selenium_assignment % cd verify_selenium.py

```

Conclusion

All Part A installation and verification tasks were completed successfully: the development environment was documented, an isolated virtual environment was created and activated, Selenium was installed and verified, a working script confirmed browser automation end-to-end, Selenium Manager's automatic driver handling was observed and evidenced across two runs, and the environment was captured in requirements.txt for reproducibility. This environment is now ready for Assignments 1–4 (locator strategies, multiple-element handling, CSS selectors, and CSS child-node selectors).